



■ CS177 Bioinformatics: Software Development and Applications

Lecture 10: Explainable AI (XAI) and Reinforcement Learning (RL)

Jie Zheng (郑杰)

PhD, Associate Professor

School of Information Science and Technology (SIST), ShanghaiTech University

Spring, 2025



Learning objectives



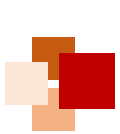
- At the end of this lecture, you should be able to:
 - Explain why we need explainable AI (XAI)
 - Give a general overview of current XAI methods
 - Describe a few well-known XAI methods (e.g. LIME, SHAP, GNNExplainer)
 - Explain the basic ideas of reinforcement learning (RL)
 - Describe how deep learning is applied to RL





Overview of Explainable AI (XAI)





Explainable AI (XAI) - what

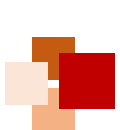


Explainability: associated with the notion of explanation as an interface between humans and a decision maker, that is, both an accurate proxy of the decision maker and comprehensible to humans

*Given an audience, an **explainable** Artificial Intelligence is one that produces details or reasons to make its **functioning clear or easy to understand.***

Arrieta et al. Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 2020, 58: 82-115.





Explainable AI (XAI) - why



- To fill the gap between the research community and business sectors or users out of the domain
- Opens the door for further model improvement and its practical utility

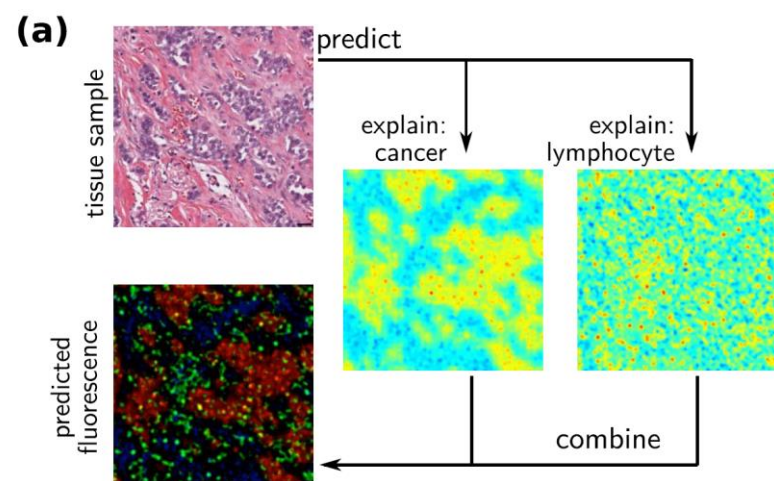
Arrieta et al. Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 2020, 58: 82-115.



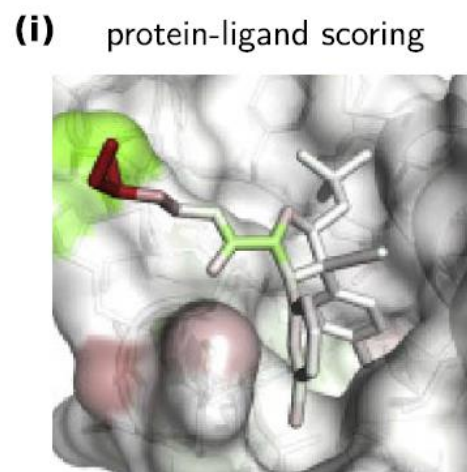
Applications of XAI



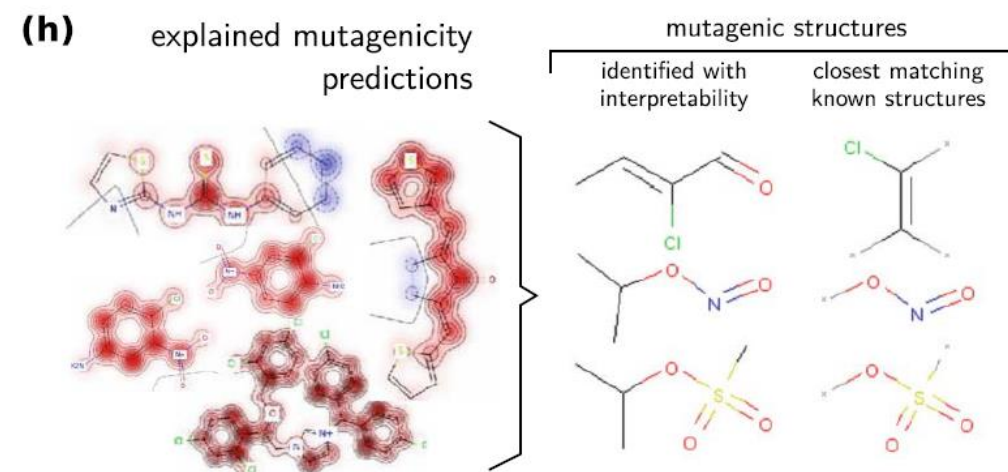
- Improve the trust of AI predictions for users
- Improve the prediction performance of AI models
- Discover new domain knowledge



Adapted from Binder et al. 2018



Adapted from Hochuli et al. 2018



Adapted from Preuer et al. 2019

Samek et al. Explaining deep neural networks and beyond: A review of methods and applications.
Proceedings of the IEEE, 2021, 109(3), 247-278.

XAI for model improvement

- **Global:** focus on providing a general understanding of a model's learned concepts and internal representations
- **Local:** explain the reasoning behind specific, singular decisions

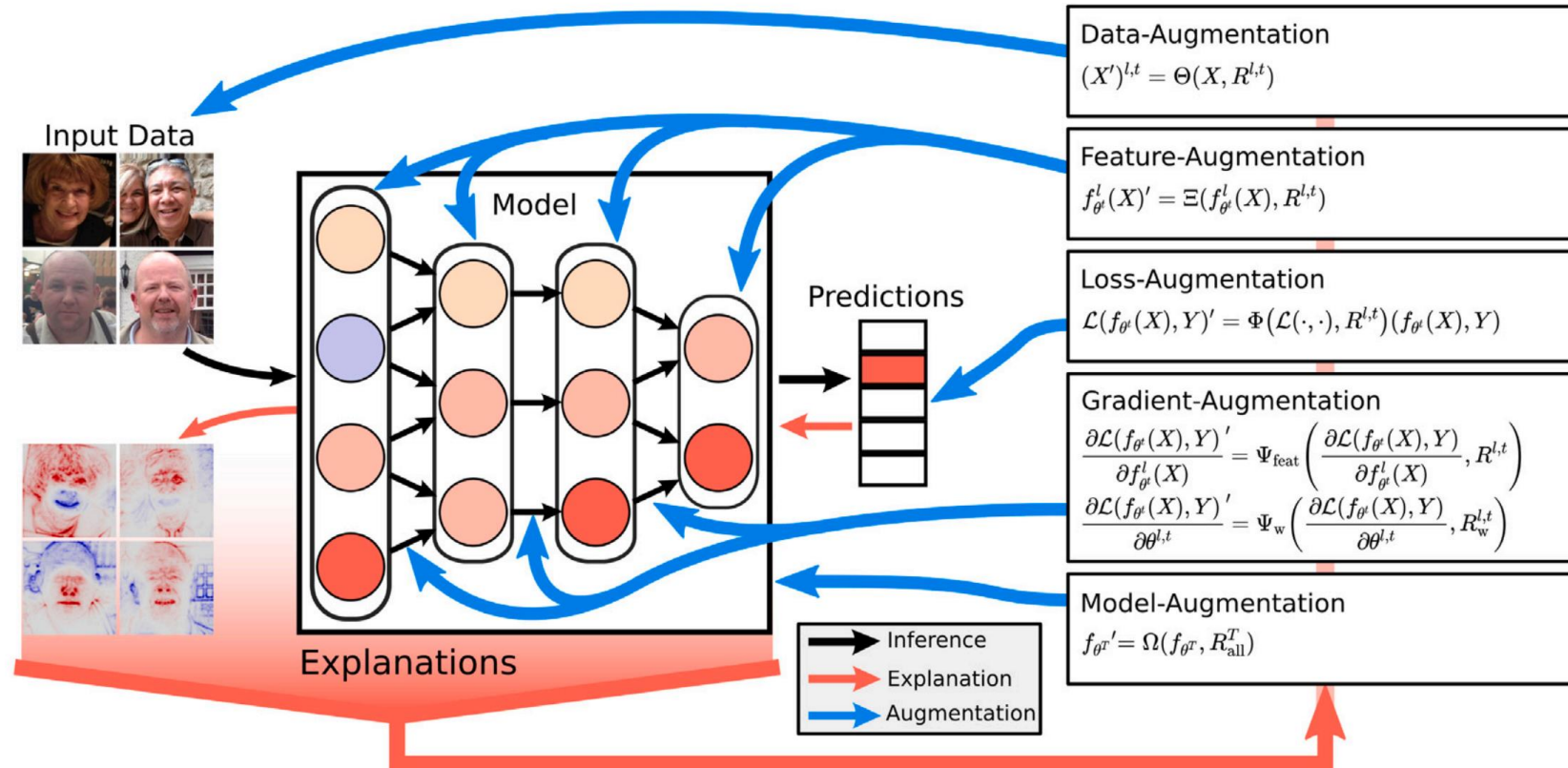
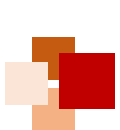


Fig. 1. Model improvement with XAI. Explanations offer information about the model decision-making and behavior, which may in turn be leveraged to improve models by augmenting different components of the training process or by adapting the trained model.



Interpretable vs. Explainable



- Interpretable machine learning (IML):
 - **Interpretable models** can be understood by a human without any other aids or methods
 - They are also called **intrinsically** or **inherently** interpretable models
 - E.g. decision tree, linear regression
- Explainable AI (XAI):
 - **Explainable models** need additional techniques to be understood by humans
 - They are also called “**black box**” models
 - E.g. random forest, neural networks
- **Interpretability** is the degree to which a model can be understood in human terms

<https://towardsdatascience.com/interperable-vs-explainable-machine-learning-1fa525e12f48>



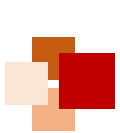


Overview of explainability

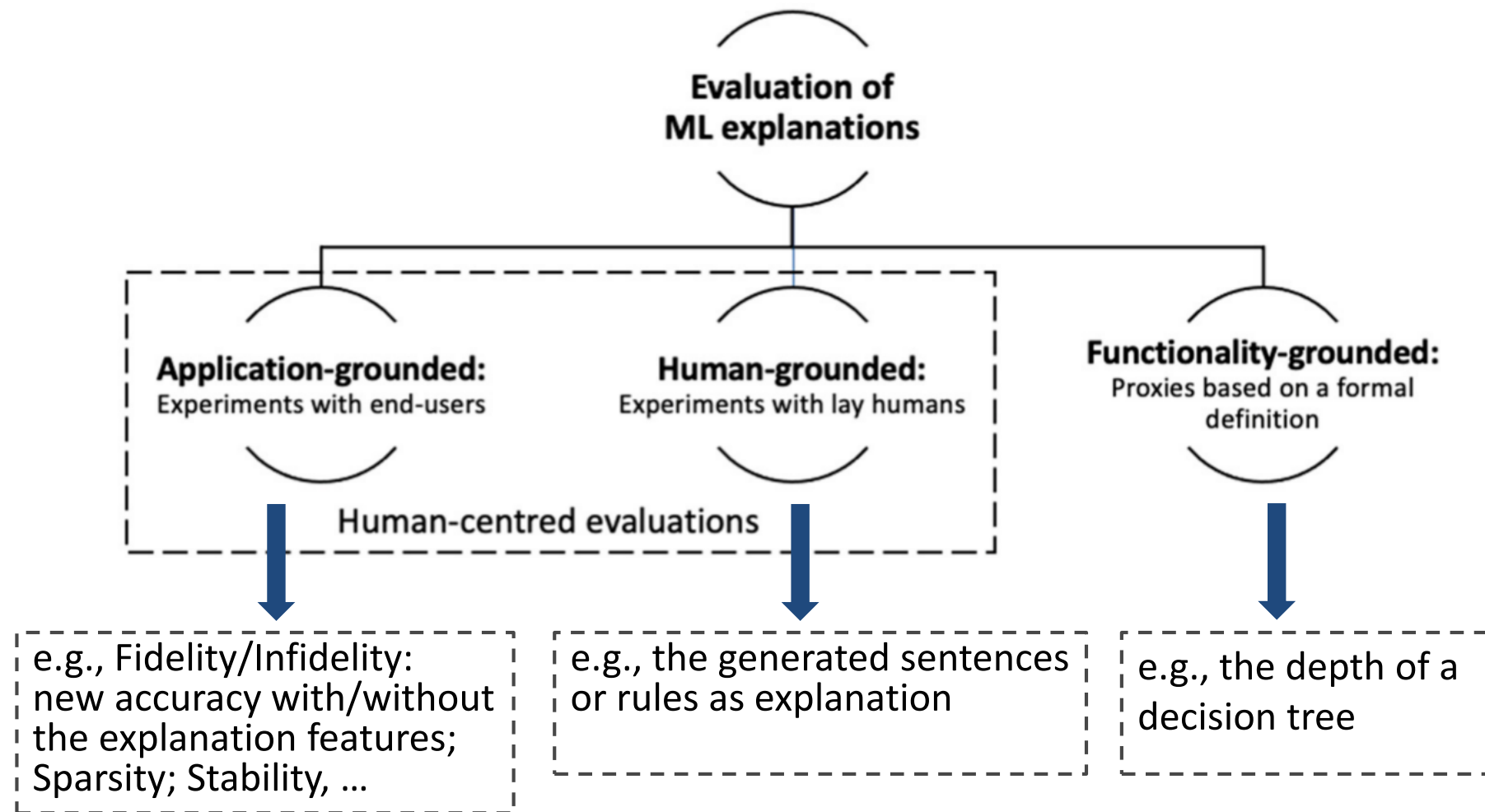


- From user' s point of view, XAI refers to the model' s decision making process
- According to different criteria, explainability of machine learning models can be divided into different categories:
 - Self-explanatory vs. post-hoc
 - Model-specific vs. model-agnostic
 - Local or instance-level vs. global or model-level





Evaluating XAI methods



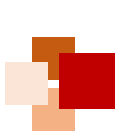
[1] Zhou et al. Evaluating the quality of machine learning explanations: A survey on methods and metrics. *Electronics*, 2021.

[2] Yuan et al. Explainability in graph neural networks: A taxonomic survey. *arXiv*, 2020.

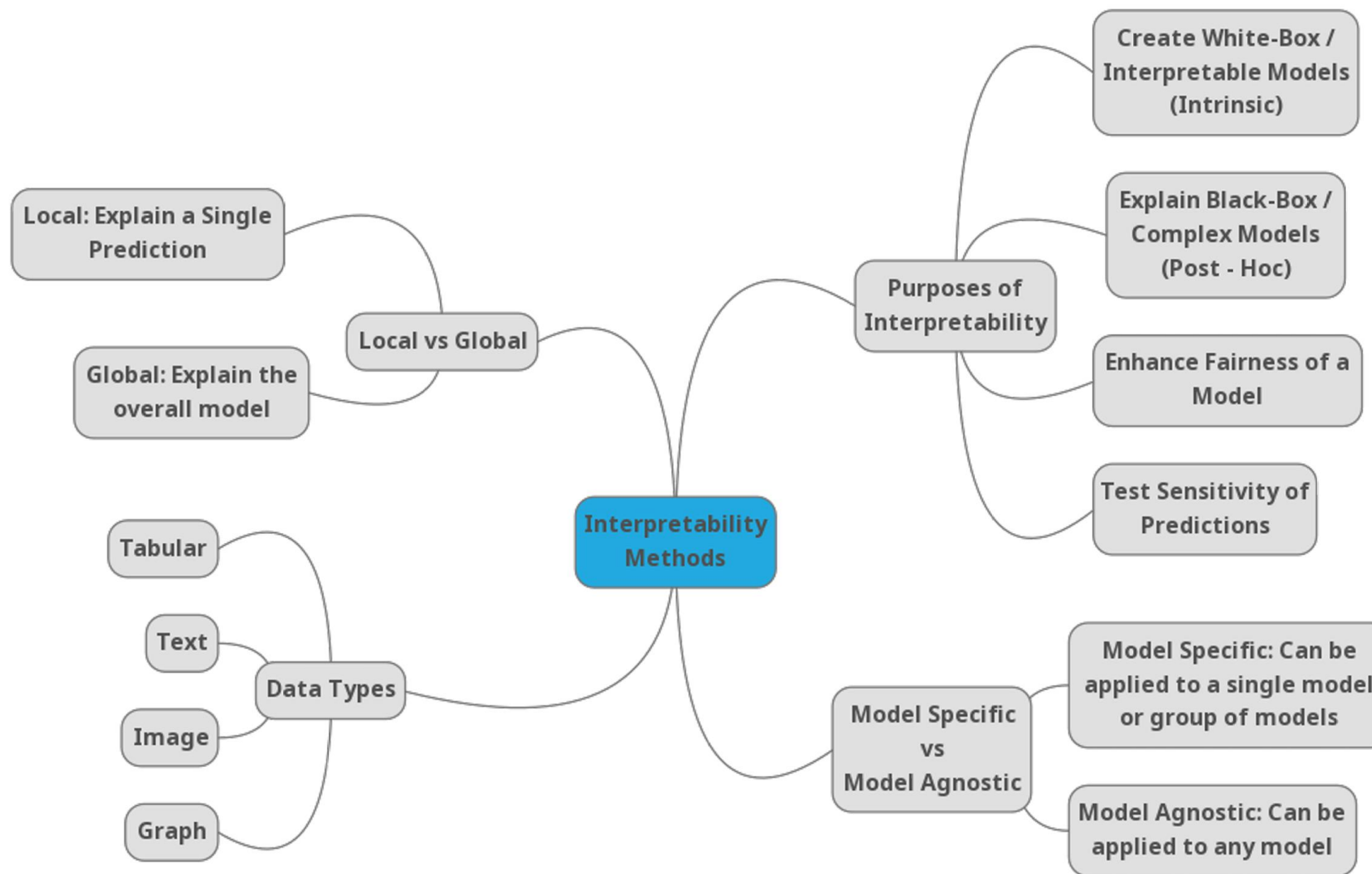


XAI Methods



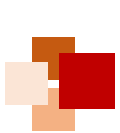


The taxonomy of XAI methods



Linardatos et al. Explainable AI: A review of machine learning interpretability methods. *Entropy*, 23(1), 18, 2021.





The taxonomy of XAI methods



Dimension 1 — Passive vs. Active Approaches

Passive	Post hoc explain trained neural networks
Active	Actively change the network architecture or training process for better interpretability

Dimension 2 — Type of Explanations (in the order of increasing explanatory power)

To explain a prediction/class by

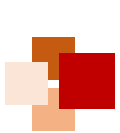
Examples	Provide example(s) which may be considered similar or as prototype(s)
Attribution	Assign credit (or blame) to the input features (e.g. feature importance, saliency masks)
Hidden semantics	Make sense of certain hidden neurons/layers
Rules	Extract logic rules (e.g. decision trees, rule sets and other rule formats)

Dimension 3 — Local vs. Global Interpretability (in terms of the input space)

Local	Explain network's <i>predictions on individual samples</i> (e.g. a saliency mask for an input image)
Semi-local	In between, for example, explain a group of similar inputs together
Global	Explain the network <i>as a whole</i> (e.g. a set of rules/a decision tree)

Zhang et al. A survey on neural network interpretability. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2021.





Overview of current XAI methods



		Local	Semi-local	Global
Passive	Rule	CEM [69], CDRPs [77], CVE ² [78], DACE [79]	Anchors [65], Interpretable partial substitution [80]	KT [81], M of N [82], NeuralRule [83], NeuroLinear [84], GRG [85], Gyan ^{FO} [86], •FZ [87], [88], Trepan [89], • [90], DecText [91], Global model on CEM [92]
	Hidden semantics	(*No explicit methods but many in the below cell can be applied here.)	—	Visualization [71], [93]–[98], Network dissection [21], Net2Vec [99], Linguistic correlation analysis [100]
	Attribution ¹	LIME [20], MAPLE [101], Partial derivatives [71], DeconvNet [72], Guided backprop [102], Guided Grad-CAM [103], Shapley values [104]–[107], Sensitivity analysis [72], [108], [109], Feature selector [110], Bias attribution [111]	DeepLIFT [112], LRP [113], Integrated gradients [114], Feature selector [110], MAME [68]	Feature selector [110], TCAV [63], ACE [115], SpRAY ³ [67], MAME [68], DeepConsensus [116]
	By example	Influence functions [73], Representer point selection [117]	—	—
Active	Rule	—	Regional tree regularization [118]	Tree regularization [119]
	Hidden semantics	—	—	“One filter, one concept” [70]
	Attribution	ExpO [120], DAPr [121]	—	Dual-net (feature importance) [122]
	By example	—	—	Network with a prototype layer [76], ProtoPNet [123]





Self-explanatory (or interpretable) models



- **Linear regression**

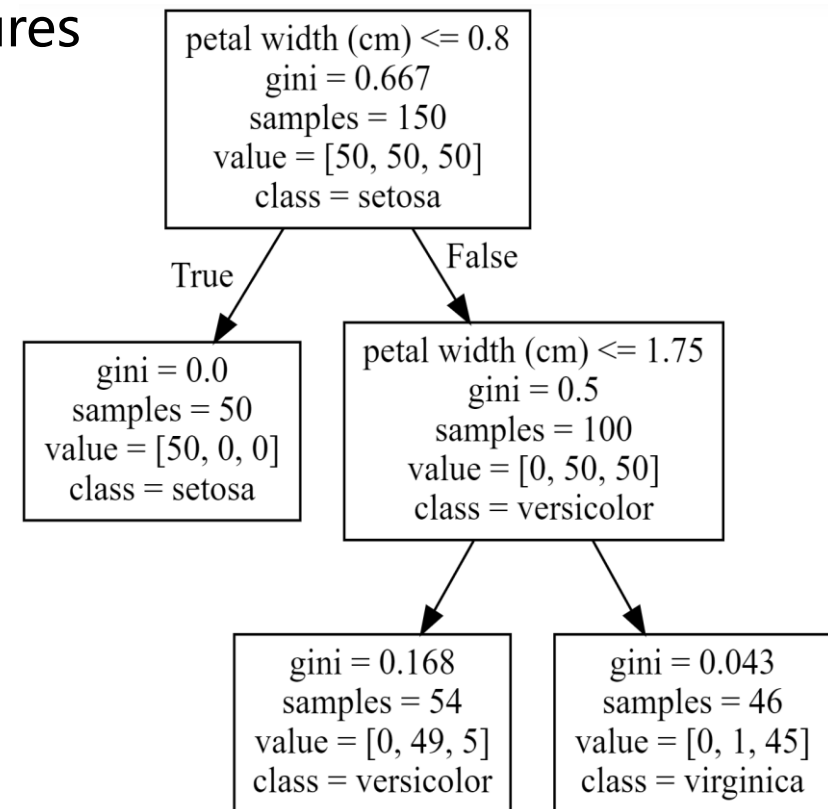
- $y = a_1x_1 + a_2x_2 + \dots + b$
- Bigger the coefficients indicate more important features

- **Decision trees**

- Each node: a feature
- Each branch: a decision
- Each leaf node: a final prediction result

- **Rule-based models**

- If-then: like a branch in the decision tree
- **AMIE (Association Rule Mining under Incomplete Evidence)**
 - Automatically learn rules from knowledge graph
 - Learn a rule for each predicted relationship, and expand the set of rules, keeping rules of high confidence for predicting new relationships

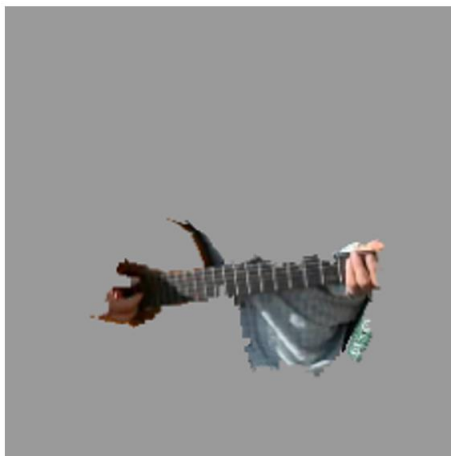


Explain based on attention/weight

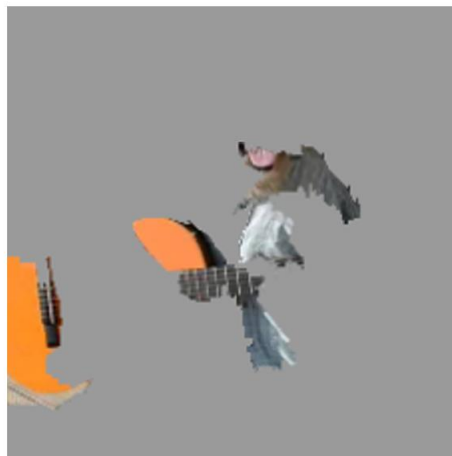
- Neural network models based on the attention mechanism
 - Assign different weights to input features
 - Learn the weights
 - Keep features of high weights as important (i.e. feature making significant contributions to the prediction results)
- Examples in AI applications:
 - Computer vision: areas of high weights in an image
 - Natural language processing: words (tokens) of high weights in an input sentence



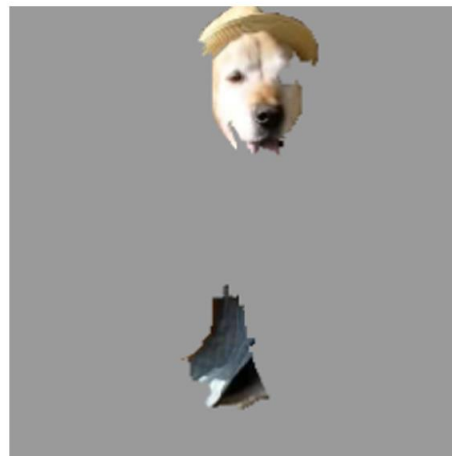
(a) Original Image



(b) Explaining *Electric guitar*



(c) Explaining *Acoustic guitar*



(d) Explaining *Labrador*

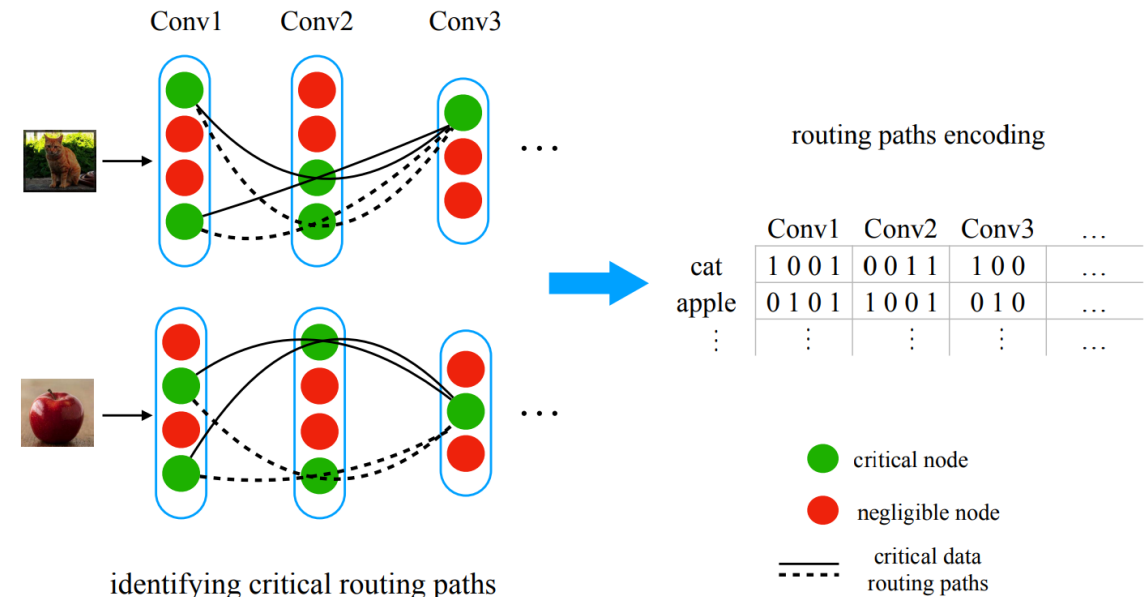
Ribeiro et al. Why should I trust you? Explaining the predictions of any classifier. *KDD*, 2016.

Post-hoc explanation

- **Model inspection methods:** Extract important structures inside the neural network, and use external algorithms to study the working mechanisms of the neural network

- **Distillation Guided Routing (DGR)**

- In the network, each layer's output channels are associated with a **scalar control gate**
- The activated channels serve as **key nodes** on the path
- The control gates learned by each layer are sequentially connected to form the **routing paths**



Yulong Wang et al. "Interpret neural network by identifying critical data routing paths", CVPR 2018.

Post-hoc explanation

- **Saliency detection:** Identifying which attributes of the input data are most relevant to the model
 - **Model Parameter Randomization Test:** Comparing the outputs of a well-trained model versus the outputs of a model with randomly initialized parameters using saliency methods
 - **Data Randomization Test:** Applying saliency methods to a model trained on a given labeled dataset versus a model trained on a dataset with randomly permuted labels

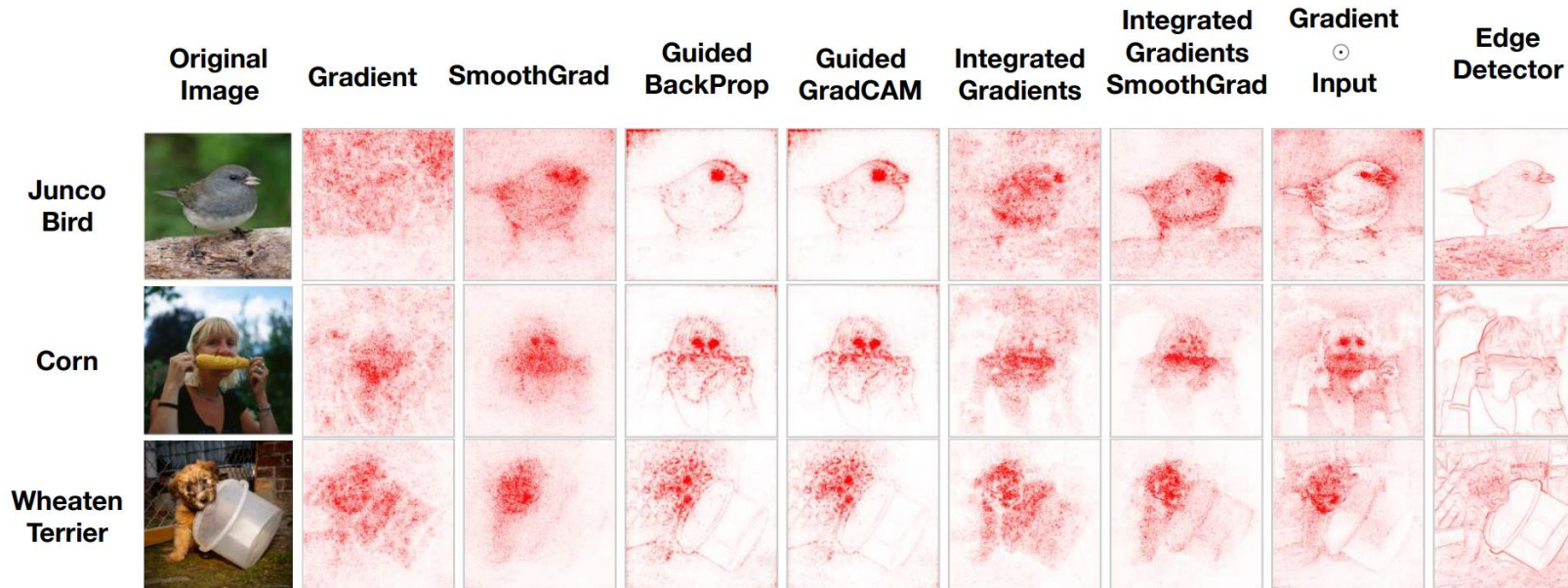
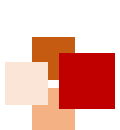


Figure. Saliency maps for some common methods compared to an edge detector



XAI based on knowledge graph (KG)



- Based on KG paths, i.e. a sequence of nodes and relationships

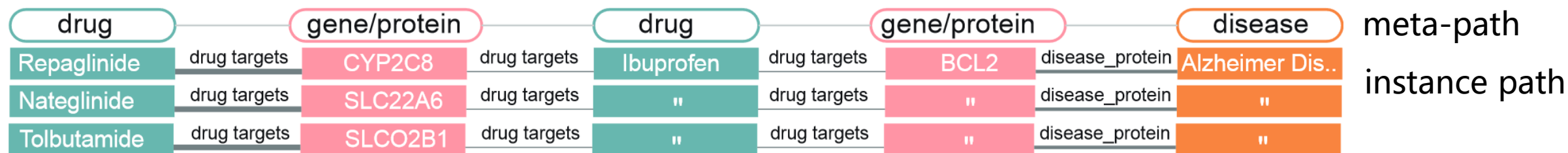
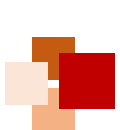


Figure. Knowledge graph pathway for drug repurposing (drug-disease interaction relationships)

Figure from:

Wang et al. Extending the Nested Model for User-Centric XAI: A Design Study on GNN-based Drug Repurposing. TVCG 2023





XAI based on knowledge graph (KG)



- Based on KG subgraphs, i.e. a combination of multiple paths
 - E.g. **SumGNN** for prediction of Drug-Drug Interactions (DDI):
Extracting closed subgraphs for drug pairs, learning subgraph representations, and making predictions

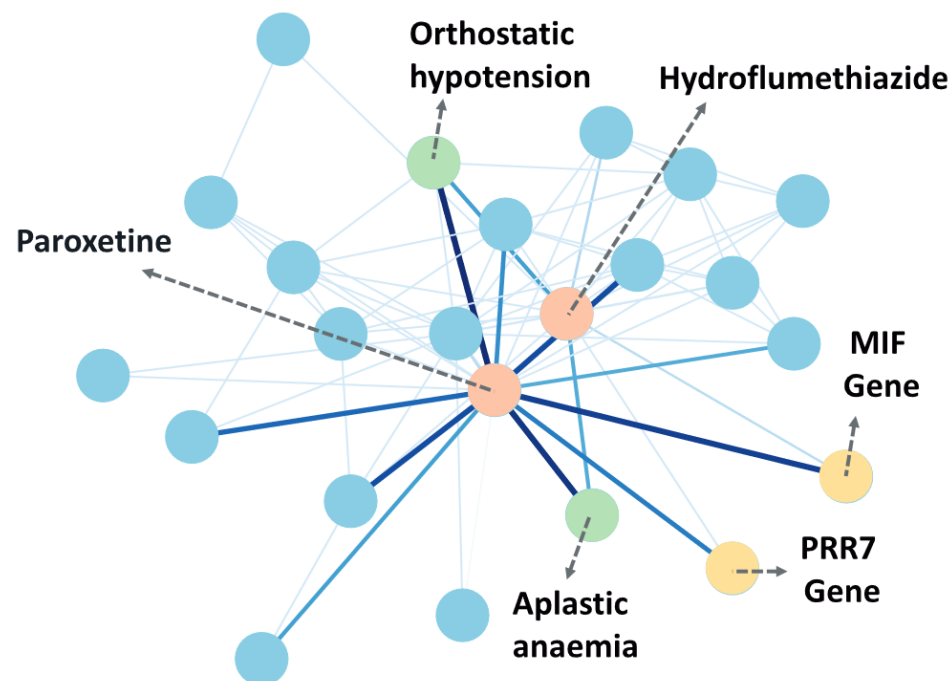


Figure from:
Yu et al (2021). SumGNN: multi-typed drug interaction prediction via efficient knowledge graph summarization. *Bioinformatics*, 37(18), 2988-2995.





XAI based on natural language processing



上海科技大学
ShanghaiTech University

- **Template-based:** Define templates and fill in the blanks of the templates as explanations
- **Generative modeling:** Using sequential models (LSTM, GRU) or Transformer-based GPT and other generative models to generate explanations in natural languages

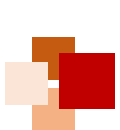


立志成才报²¹国裕民



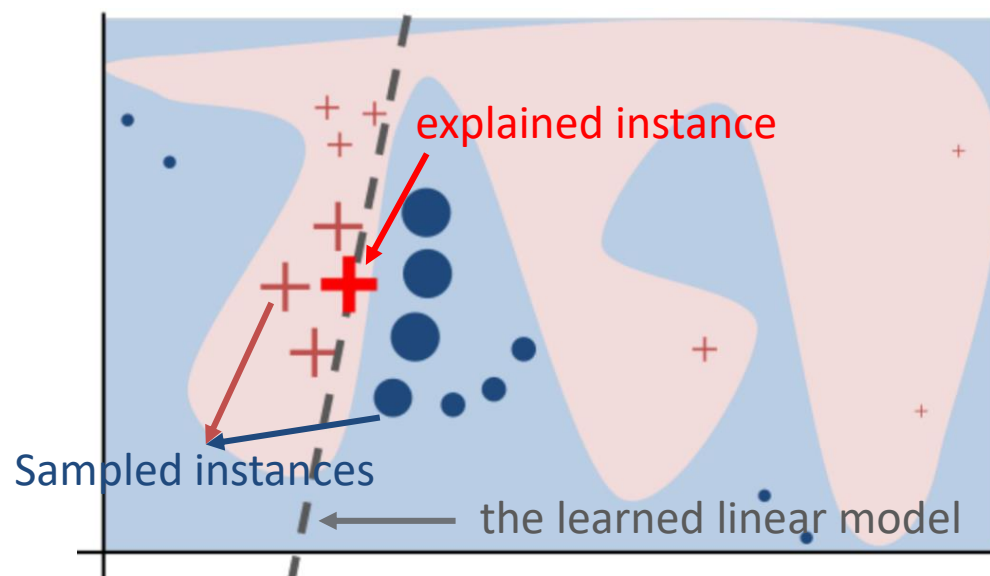
XAI Examples





- **LIME (Local Interpretable Model-agnostic Explanations)**

Idea: sample instances to learn a linear model that could **locally** approximate the original predictions



Prediction probabilities

atheism	0.58
christian	0.42

atheism

christian

Posting	0.15
Host	0.14
NNTP	0.11
edu	0.04
have	0.01
There	0.01

Text with highlighted words

From: johnchad@triton.unm.edu (jchadwic)

Subject: Another request for Darwin Fish

Organization: University of New Mexico, Albuquerque

Lines: 11

NNTP-Posting-Host: triton.unm.edu

Hello Gang,

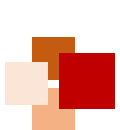
There have been some notes recently asking where to obtain the DARWIN fish.

This is the same question I have and I have not seen an answer on the

net. If anyone has a contact please post on the net or email me.

Ribeiro et al. Why should I trust you? Explaining the predictions of any classifier. *KDD* 2016.

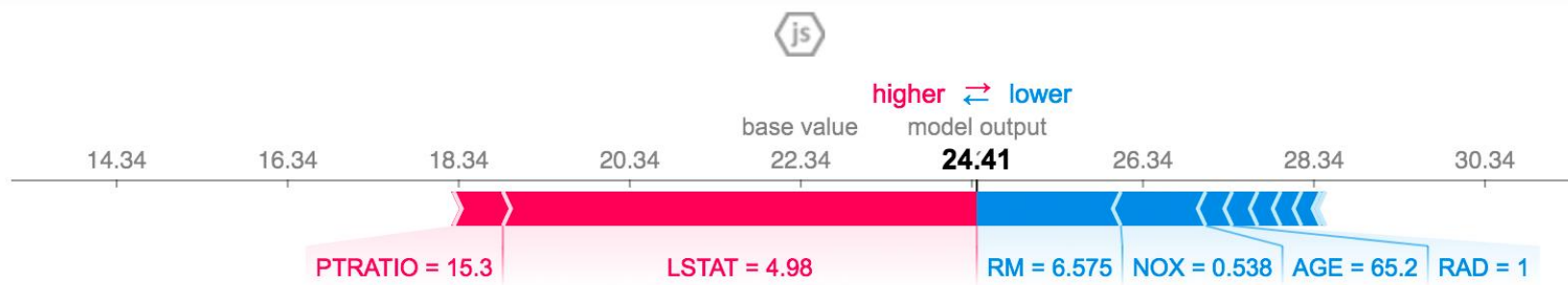




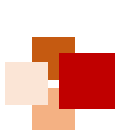
SHAP



- **SHAP**, or **SHapley Additive exPlanations**, is a game theoretic approach to explain the output of any machine learning model
 - It connects **optimal credit allocation** with **local explanations** using the classic **Shapley values** from cooperative game theory and their related extensions
- In machine learning, a **Shapley value** is the contribution of a feature value to difference between the actual prediction and the mean prediction
 - Introduced in 1953 by Lloyd Shapley (who won Nobel Prize in Economics, 2012), one of the fathers of Game Theory
 - **Intuition**: Sometimes a player's value **in a team** could be greater than his/her value **on their own**. For machine learning, we can view the features as the "players" and the outcome of the model as the "game's result"
 - Given that without any features we would just predict an average value, once we bring the first feature in, how much our prediction would change compared to the average?



Lundberg & Lee. A unified approach to interpreting model predictions. *NeurIPS* 2017.

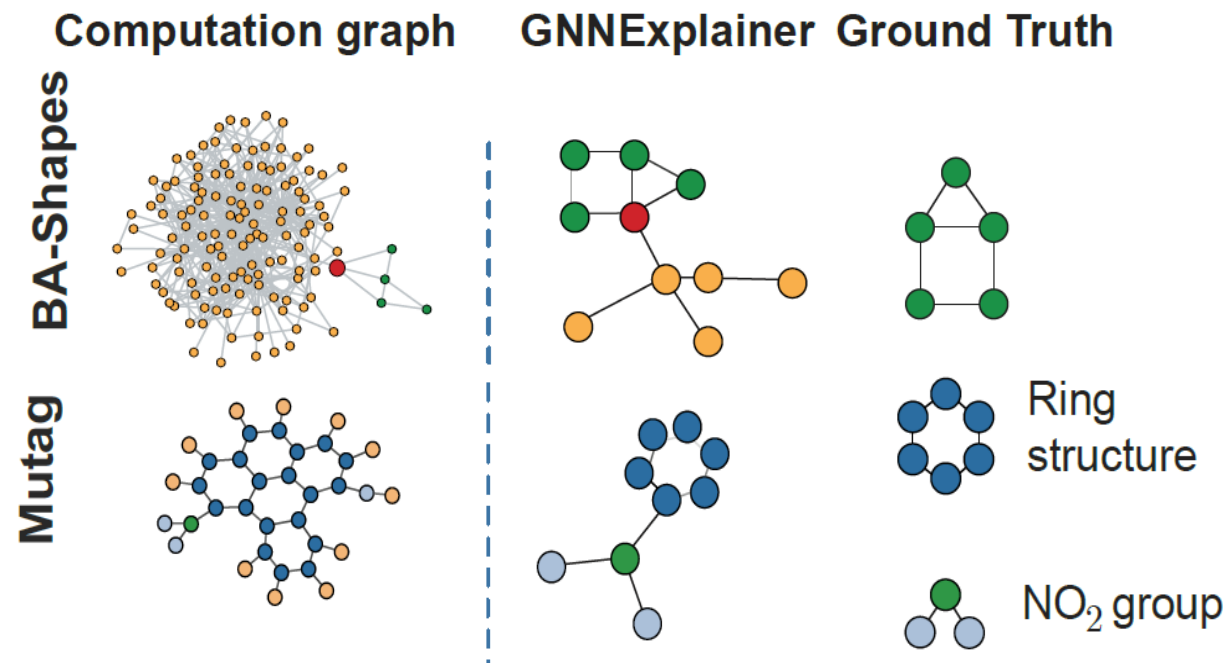
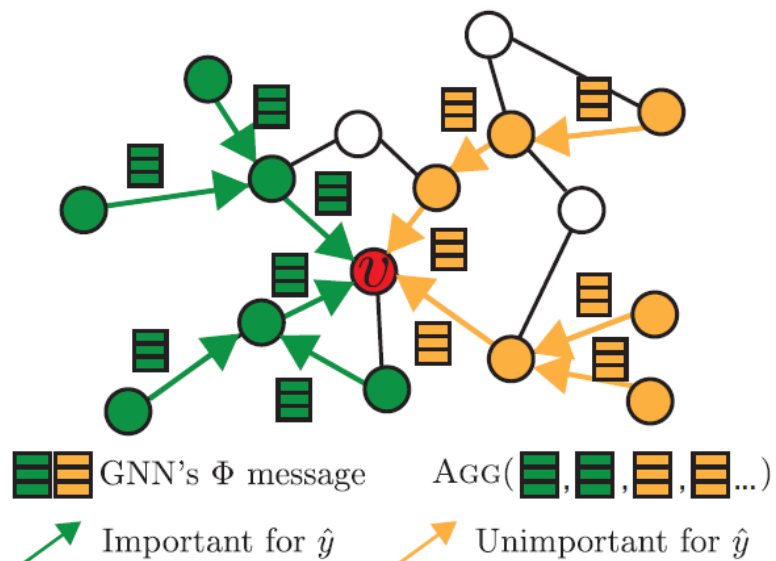


GNNExplainer



上海科技大学
ShanghaiTech University

Learning a subgraph to approximate the predictions of a GNN model which uses a whole graph



Rex Ying et al. GNNExplainer: Generating explanations for graph neural networks. *NeurIPS*, 2019.



GNNExplainer: key idea

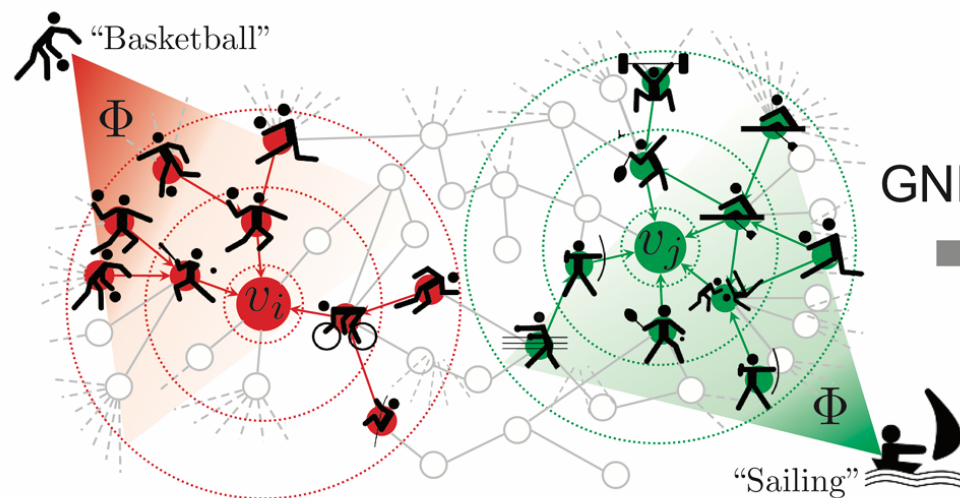


Key idea:

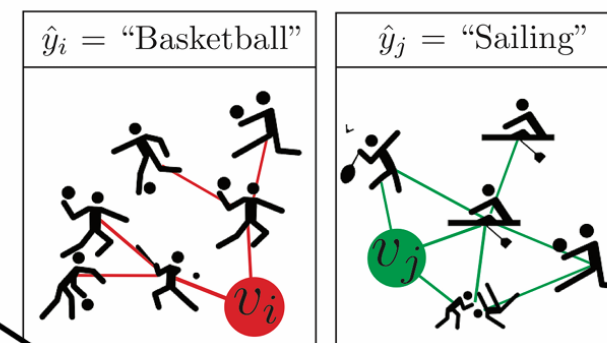
- Summarize where in the data the model “looks” for evidence for its prediction
- Find a small subgraph **most influential** for the prediction



GNN model training and predictions



Explaining GNN's predictions



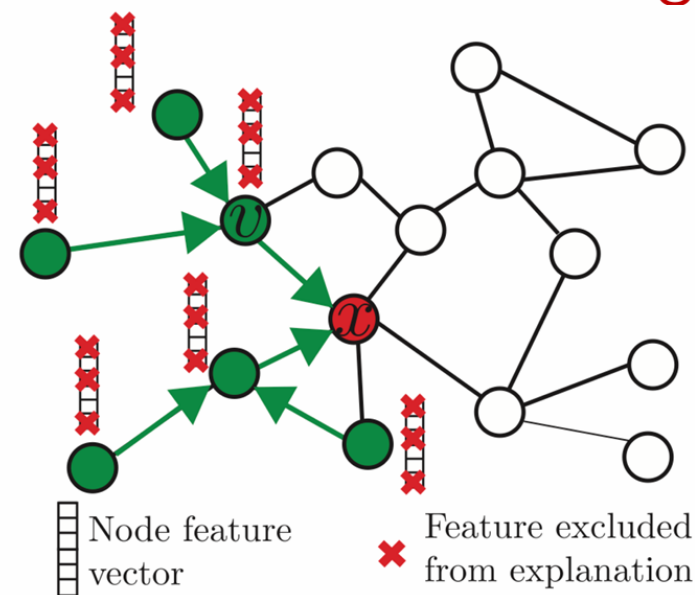
GNNExplainer

Approach to generate explanations for graph neural networks based on **counterfactual reasoning**

■ GNNExplainer: key idea



- **Input:** Given prediction $f(x)$ for node/link x
- **Output:** Explanation, a small subgraph M_x together with a small subset of node features:
 - M_x is most influential for prediction $f(x)$
- **Approach:** Learn M_x via **counterfactual reasoning**
 - **Intuition:** If removing v from the graph strongly decreases the probability of prediction $\Rightarrow v$ is a good counterfactual explanation for the prediction





GNNExplainer: Explain by Mutual Information



- **Mutual information (MI)**

- A measure of the mutual correlation between two random variables
- Good explanation should have **high correlation** with model prediction
- Relation to entropy:

$$MI(X; Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

- GNNExplainer' s **objective**:

- Maximize the MI between label Y and explanation (the subgraph and subset of features):

$$\max_{G_S} MI(Y; (A_S, X_S)) = H(Y) - H(Y|A = A_S, X = X_S^F)$$

- where G_S with adjacency matrix A_S is a subgraph of the input graph with adjacency matrix A , and X_S^F is the set of features for G_S





Reinforcement Learning (RL)





Reinforcement learning (RL)



- A new class of problems: Reward-based
 - E.g. autonomous driving
 - What is my next move?
 - Reaching the destination with minimum cost





Reinforcement learning (RL)

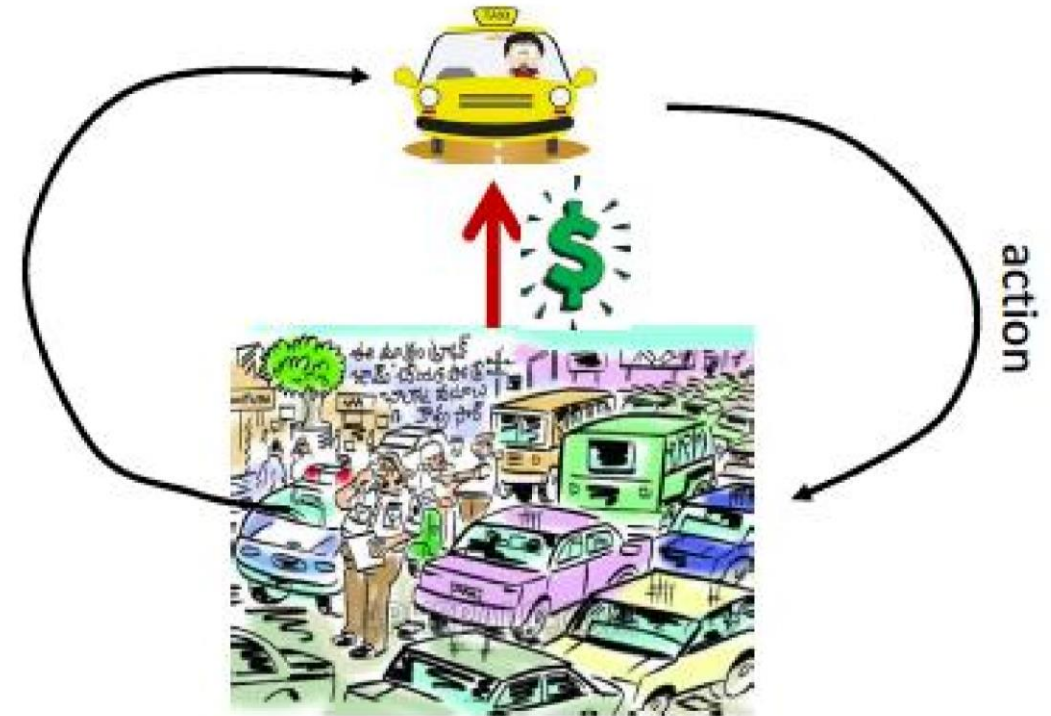


- Common theme: control problems where
 - Your actions beget rewards
 - Win a Go game
 - Make money by investing
 - Get home sooner
 - But not deterministically
 - A world out there that is not predictable
- From experience of **belated/delayed** rewards, you must learn to act rationally



RL problem setting

- An agent operates in an environment
- The agent takes actions that
 - affect the environment
 - change in a somewhat unpredictable way
 - affect the agent's situation
- The agent also receives rewards
 - which may be apparent immediately
 - or not apparent for a very long time



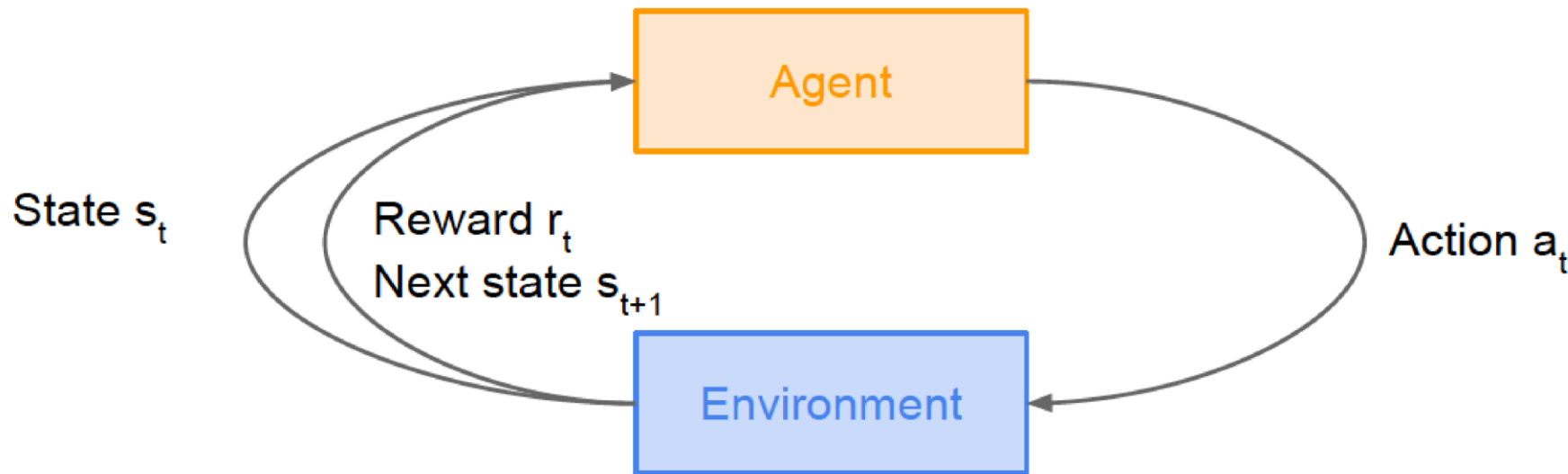
Problem to solve:

How must the agent behave to maximize its rewards?





Markov Decision Process (MDP)



- Markov assumption:
 - All relevant information is encapsulated in the current state
 - i.e. the policy, reward, and transitions are all **independent** of past states given the current state
- Assume a fully observable environment, i.e. the state can be observed directly





Formal definition of MDP



A *Markov Decision Process* is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P}$ is a state transition probability matrix,
 $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$
- $\mathcal{R}$ is a reward function, $\mathcal{R}_s^a = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$
- γ is a discount factor $\gamma \in [0, 1]$.



Formal definition of MDP

- At time step $t = 0$, environment samples initial state $s_0 \sim p(s_0)$
- Then, for $t = 0$ until done:
 - Agent selects action a_t
 - Environment samples reward $r_t \sim R(\cdot | s_t, a_t)$
 - Environment samples next state $s_{t+1} \sim P(\cdot | s_t, a_t)$
 - Agent receives reward r_t and next state s_{t+1}
- A **policy** π is a function from S to A that specifies what action to take in each state
- **Objective**: find policy π^* that maximizes cumulative discounted reward: $\sum_{t \geq 0} \gamma^t r_t$

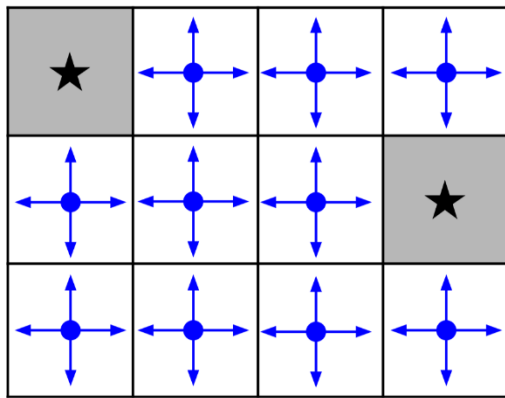
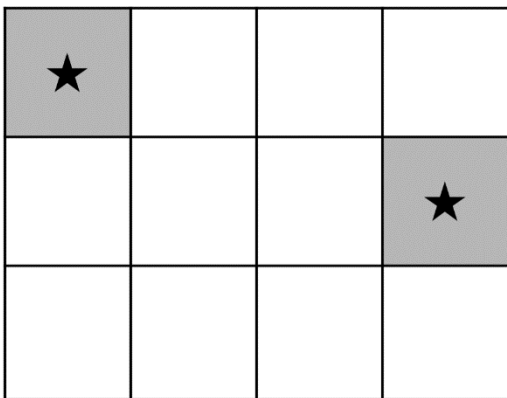


A simple MDP: Grid World

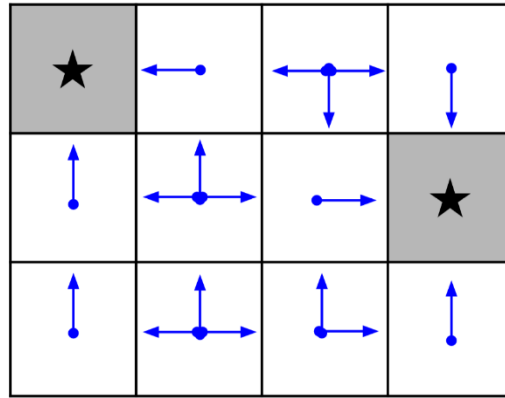
- Set a negative “reward” for each transition
- **Objective:** reach one of terminal states (greyed out) in the **least** number of actions

- actions = {
1. right
 2. left
 3. up
 4. down
- }

states



Random Policy



Optimal Policy





Problems in MDP



- **Planning**: Given a complete MDP as input, compute a policy with optimal expected return
 - Goal: maximize the expected return, $R = E_{p(\tau)}[r(\tau)]$
 - The expectation is over both the environment's dynamics and the policy, but we only have control over the policy
- **Learning**: Given samples of trajectories of an **unknown** MDP
 - **Prediction**: estimate the expected return given a policy
 - **Control**: find the optimal policy that maximizes the expected return



Value function and Q-value function

- Following a policy produces sample trajectories (or paths) $s_0, a_0, r_0, s_1, a_1, r_1, \dots$
- How good is a state?

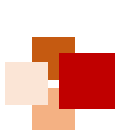
The **value function** at state s is the expected cumulative reward from following the policy from state s :

$$V^\pi(s) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid s_0 = s, \pi \right]$$

- How good is a state-action pair?

The **Q-value function** at state s and action a is the expected cumulative reward from taking action a in state s and then following the policy:

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid s_0 = s, a_0 = a, \pi \right]$$



Bellman equation



- The **optimal Q-value function** Q^* is the maximum expected cumulative reward achievable from a given (state, action) pair:

$$Q^*(s, a) = \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid s_0 = s, a_0 = a, \pi \right]$$

- Q^* satisfies the following **Bellman equation**:

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

- Intuition:** If the optimal state-action values for the next time-step $Q^*(s', a')$ are known, then the optimal strategy is to take the action that maximizes the expected value of $r + \gamma Q^*(s', a')$
- The optimal policy $\pi^* = \operatorname{argmax}_a Q^*(s, a)$





Q-Value Iteration algorithm



- **Value Iteration algorithm** uses the Bellman equation for an iterative update to estimate the optimal state value
 - But Q-values are more useful, as they give us optimal policies
- An iterative algorithm similar to the value iteration algorithm was found by Bellman, which is called **Q-Value Iteration algorithm**

$$Q_{i+1}(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q_i(s', a') | s, a \right]$$

- Q_i will converge to Q^* as $i \rightarrow \infty$
- **Problem:** Not scalable
 - Must compute $Q(s, a)$ for every (state, action) pair
 - Often computationally infeasible to compute for the entire state space
- **Solution:** Use a function approximator to estimate $Q(s, a)$, e.g. a neural network





Q-learning



- **Q-learning**: Use a function approximator to estimate the action-value function

$$Q(s, a; \theta) \approx Q^*(s, a)$$

function parameters (weights)

- Learn $Q(s, a)$ values as you go
 - Receive a sample (s, a, s', r)
 - Consider your old estimate: $Q(s, a)$
 - Consider your new sample estimate
 - Incorporate the new estimate into a running average
- If the function approximator is a **deep neural network**, then it is called **deep Q-learning**



Q-learning properties



- Q-learning converges to optimal policy, even if you are acting suboptimally
 - It is called **off-policy learning**, as the policy being trained is not necessarily the one being executed
- **Caveats:**
 - You have to **explore** enough
 - You have to eventually make the learning rate small enough, but not decrease it too quickly
 - In the limit, it doesn't matter how you select actions



- In this lecture, we have learned
 - Why explained AI (XAI) is important and useful
 - What are the main categories of XAI methods
 - How do a few example XAI methods work (i.e. LIME, SHAP, GNNExplainer)
 - What is the rationale behind Reinforcement Learning (RL)
 - How Markov Decision Process (MDP) is used to model the RL problem setting
 - How the MDP problem is reduced to Q-learning
 - How deep neural network is used in the deep Q-learning





References



- Christoph Molnar, Book “Interpretable Machine Learning: A Guide for Making Black Box Models Explainable” (2024-07-31)
 - <https://christophm.github.io/interpretable-ml-book/>
- Chapter 18 of Aurélien Géron’ s book “Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow” (Ed. 2) , O’ Reilly, 2019 (on Reinforcement Learning)
- Zhang et al. A survey on neural network interpretability. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2021.
- Ribeiro et al. Why should I trust you? Explaining the predictions of any classifier. *KDD*, 2016. (**LIME**)
- Lundberg & Lee. A unified approach to interpreting model predictions. *NeurIPS* 2017. (**SHAP**)
- Rex Ying et al. GNNExplainer: Generating explanations for graph neural networks. *NeurIPS*, 2019. (**GNNExplainer**)

